

Lab 4: VibeCoding

Vibe Coding Lab | Lab 4 | 2026-10-07

Aim

Fix the maze runner movement defect and use Vibe Coding to add the requested shortest-path hint, fog of war, persistent timer leaderboard, and difficulty selection.

Procedure

I reviewed the starter README, corrected player collision detection, implemented the four listed features, and ran Python compilation plus headless Pygame assertions. I also recorded a genuine 10-second capture of the running game after the changes. The original, unmodified Maze Runner was recorded before changes. The genuine 11.9-second before video is included in this submission.

Task 1: Fix wall collision detection

Command(s) used:

```
python main.py
python -m compileall game main.py
Headless Pygame assertions for closed-wall and open-passage movement
```

Observation: I found that the collision check rejected positions outside the board but did not check internal cell walls. I added checks against each cell boundary and verified repeated 3-pixel movement stops at a closed wall while still passing through a connected opening.

Task 2: Shortest-path hint with H toggle

Command(s) used:

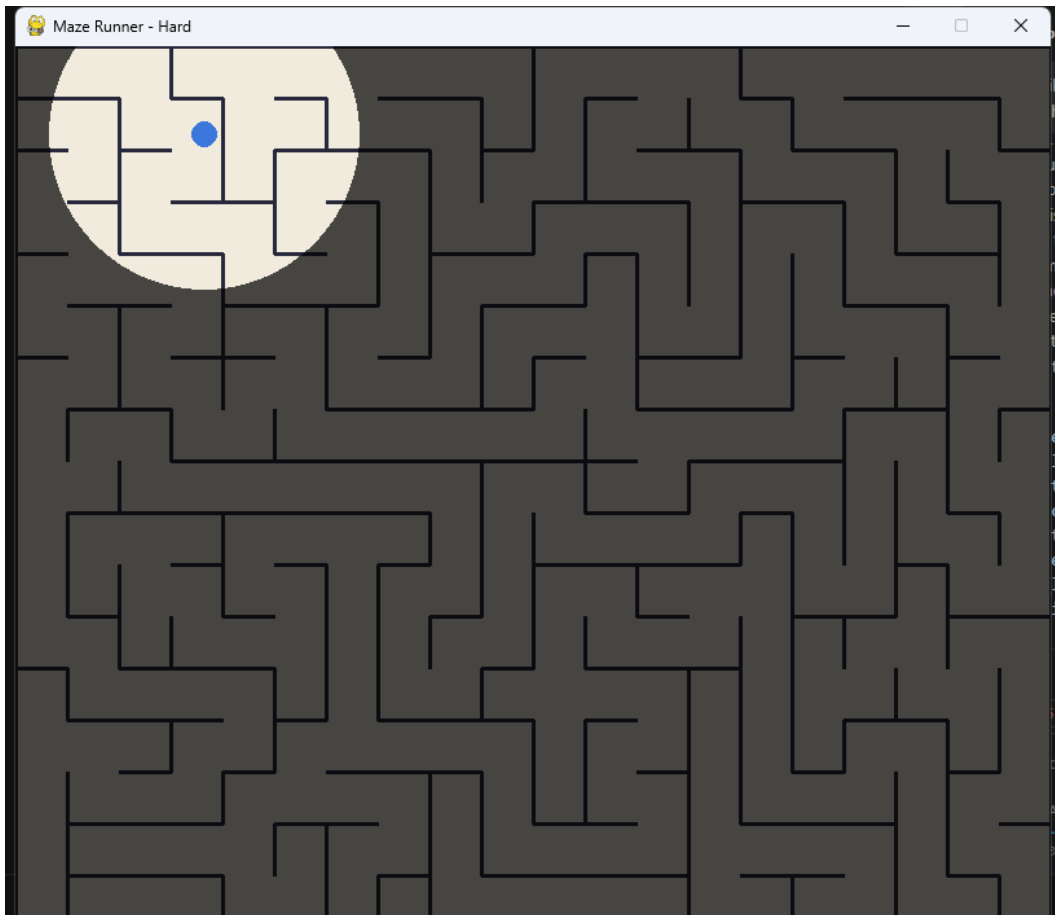
```
python main.py
Headless Pygame assertions for BFS path endpoints and H-key toggle
```

Observation: The breadth-first search produced a connected route from the upper-left start to the lower-right exit for each supported board size. I also checked that H switches the highlighted path on and off.

Task 3: Fog of war

Command(s) used:

```
python main.py  
Headless Pygame pixel assertions for the three-cell reveal radius
```



Observation: I corrected the transparent cutout so it clears the fog surface itself; blitting a fully transparent helper surface had not revealed the maze. The captured live game shows the player inside the illuminated area, and pixel checks confirmed that the nearby area stays visible while the area outside the radius is dimmed.

Task 4: Timer leaderboard saved to JSON

Command(s) used:

```
python main.py  
Headless Pygame assertions for sorted top-five JSON persistence
```

Observation: I tested recording completion times against a temporary JSON file. The entries were sorted fastest first, capped at five, and read back from disk in the same order; a dedicated temporary leaderboard file verified that completion times are sorted fastest first, retained to five entries, and persisted to JSON. The repository leaderboard is kept empty until a real player finishes a maze.

Task 5: Easy, Medium, and Hard difficulty tiers

Command(s) used:

```
python main.py  
Headless Pygame assertions for all three board dimensions
```

Observation: The selection menu provides Easy (10 x 8), Medium (15 x 13), and Hard (20 x 18). I instantiated each setting and confirmed the game engine generated a valid start-to-exit route for every size.

Conclusion

Compilation and targeted runtime checks passed for collision handling, BFS hints, fog visibility, sorted leaderboard persistence, and all difficulty dimensions. The original starter-game recording is included as the before video. The after video will be captured from a fresh updated-game session after the student records it.